



DrK: Breaking Kernel Address Space Layout Randomization with Intel TSX

Yeongjin Jang, Sangho Lee, and Taesoo Kim
Georgia Institute of Technology, August 3, 2016

J U L Y 3 0 - A U G U S T 4 , 2 0 1 6 / M A N D A L A Y B A Y / L A S V E G A S

Outline

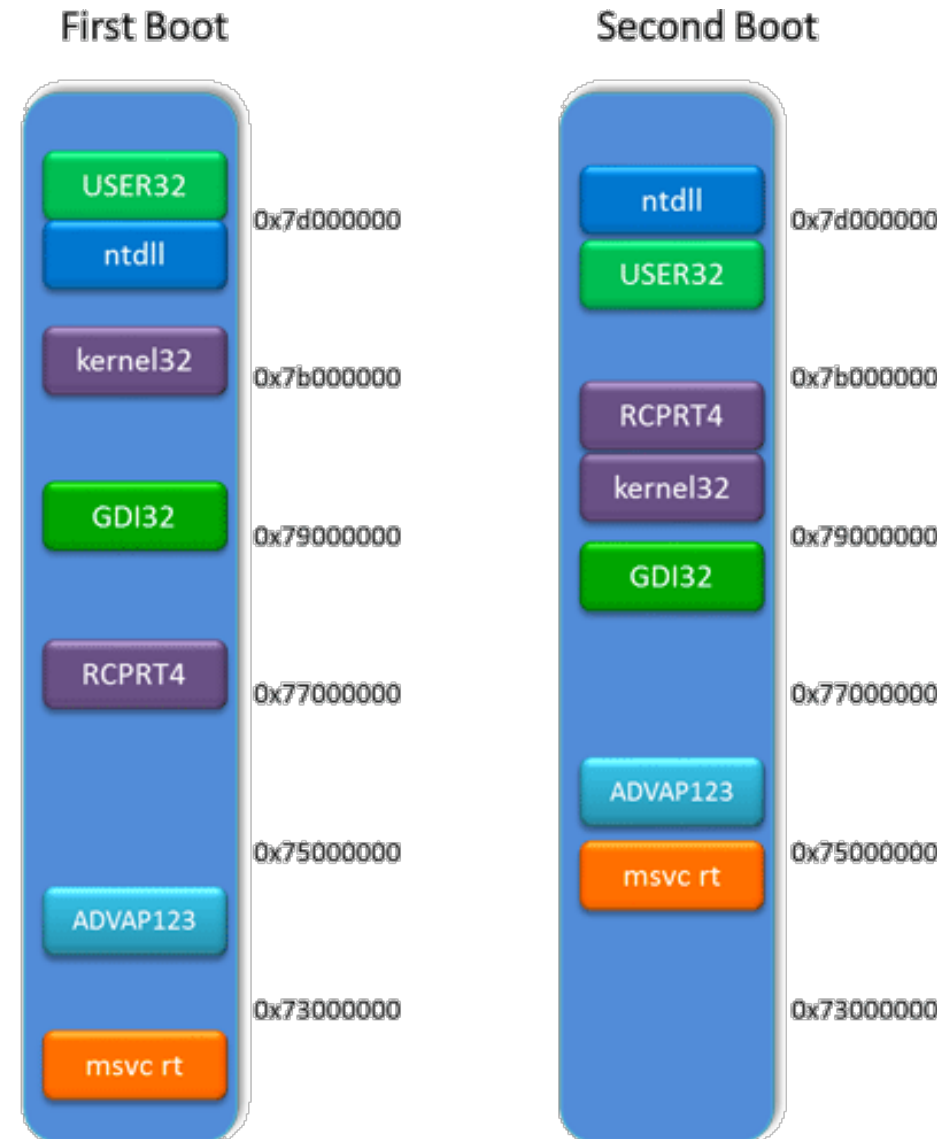
- KASLR Background
- TLB Side Channel Attack on KASLR
- Attacking TLB Side Channel with Intel TSX
- Attacking various OSes
- Root Cause Analysis
- Discussions
- Conclusion

Outline

- KASLR Background
- TLB Side Channel Attack on KASLR
- Attacking TLB Side Channel with Intel TSX
- Attacking various OSes
- Root Cause Analysis
- Discussions
- Conclusion

Kernel Address Space Layout Randomization (KASLR)

- A statistical mitigation for memory corruption exploits
- Randomize address layout per each boot
 - Efficient (<5% overhead)
- Attacker should guess where code/data are located for exploit.
 - In Windows, a successful guess rate is 1/8192.



Example: Linux

- To escalate privilege to root through a kernel exploit, attackers want to call `commit_creds(prepare_kernel_cred(0))`.

```
// full-nelson.c
static int __attribute__((regparm(3)))
getroot(void * file, void * vma)
{
    commit_creds(prepare_kernel_cred(0));
    return -1;
}

// https://blog.plenz.com/2013-02/privilege-escalation-kernel-exploit.html
int privesc(struct sk_buff *skb, struct nlmsghdr *nlh)
{
    commit_creds(prepare_kernel_cred(0));
    return 0;
}
```

Example: Linux

- Kernel symbols are hidden to non-root users.

```
blue9057@pt ~ $ cat /proc/kallsyms | grep 'commit_creds\|prepare_creds'
0000000000000000 T commit_creds
0000000000000000 T prepare_creds
```

- KASLR changes kernel symbol addresses every boot.

```
blue9057@pt ~ $ sudo cat /proc/kallsyms | grep 'commit_creds\|prepare_creds'
fffffffffaa0a3bd0 T commit_creds
fffffffffaa0a3e20 T prepare_creds
```

1st Boot

```
blue9057@pt ~ $ sudo cat /proc/kallsyms | grep 'commit_creds\|prepare_creds'
fffffffff850a3bd0 T commit_creds
fffffffff850a3e20 T prepare_creds
```

2nd Boot

Example: tpwn - OS X 10.10.5

Kernel Privilege Escalation Vulnerability

- [CVE-2015-5864] IOAudioFamailiy allows a local user to obtain sensitive kernel memory-layout information via unspecified vectors.

```
char found = 0;
DO_TIMES(ALLOCS) {
    char* data = read_kern_data(heap_info[ctr].port);
    if (!found && memcmp(data,vz,1024 - 0x58)) {
        kslide = (*(uint64_t*)((1024-0x58+(char*)data))) - kslide ;
        found=1;
    }
}
if (!found) {
    exit(-3);
}
printf("leaked kaslr slide, @ 0x%016llx\n", kslide);
```

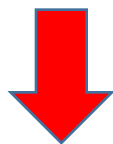
Bypassing KASLR is
required...



KASLR Makes Attacks Harder

- KASLR introduces an additional bar to exploits
 - Finding an information leak vulnerability

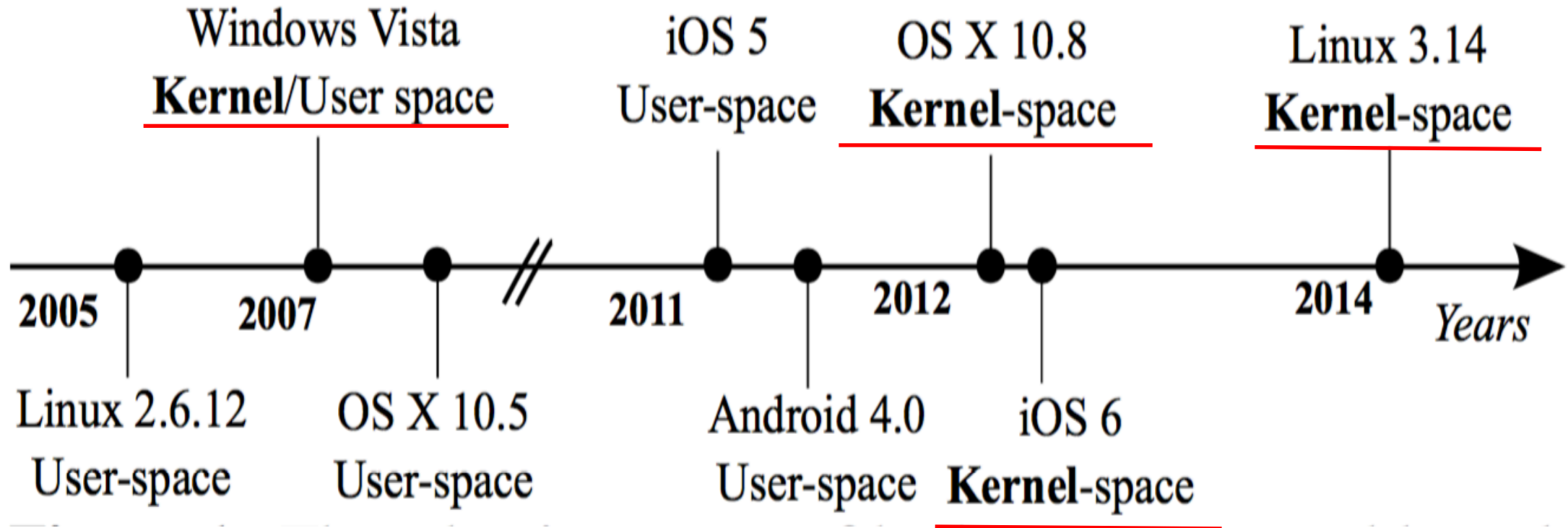
$\text{Pr}[\exists \text{ Memory Corruption Vuln}]$



$\text{Pr}[\exists \text{ information_leak}] \times \text{Pr}[\exists \text{ Memory Corruption Vuln}]$

- Both attackers and defenders aim to detect info leak vulnerabilities.

Popular OSes Adopted KASLR



Outline

- KASLR Background
- TLB Side Channel Attack on KASLR
- Attacking TLB Side Channel with Intel TSX
- Attacking various OSes
- Root Cause Analysis
- Discussions
- Conclusion

Is there any other way than info leak?

- Practical Timing Side Channel Attacks Against Kernel Space ASLR (Hund et al., Oakland 2013)
 - A **hardware-level** side channel attack against KASLR
 - **No** information leak vulnerability in OS is required

TLB Timing Side Channel

- If accessed a kernel address from the user space

```
blue9057@pt ~ $ ./access_address 0xffffffff80000000  
Accessing address 0xffffffff80000000  
[1] 677 segmentation fault (core dumped) ./access_address 0xffffffff80000000
```

- Mapped address: Access violation, Page fault
- Unmapped address: Invalid address, Page fault

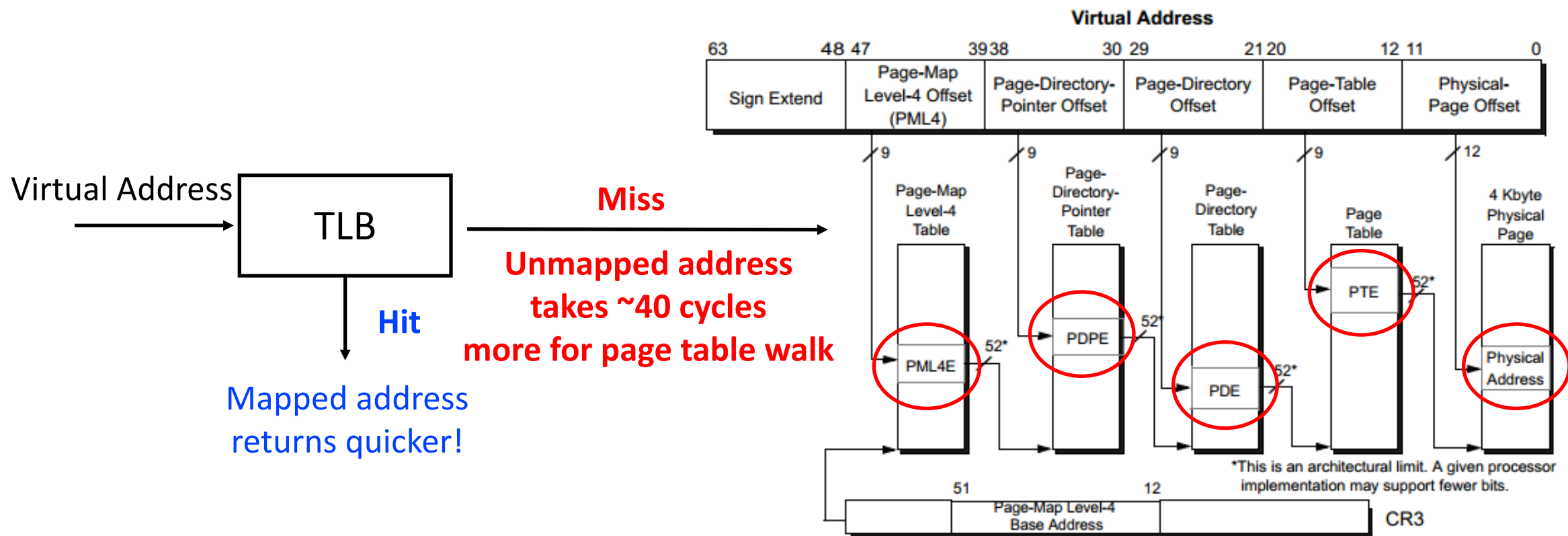
TLB Timing Side Channel

- If an **unmapped** kernel address is accessed
 1. Try to get page table entry through page table walk
 2. There is no page table entry found, generate page fault!

TLB Timing Side Channel

- If a **mapped** kernel address is accessed
 1. Try to get page table entry through page table walk
 2. **Cache the entry to TLB**
 3. Check page privilege level ($3 < 0$), generate page fault!

TLB Timing Side Channel



TLB Timing Side Channel

- Measuring the time in an exception handler

```
uint64_t time_begin, time_diff;
__try
{
    // a kernel address
    int *p = (int*)0xffffffff80000000;
    time_begin=__rdtscp();
    *p = 0;
}
__except(EXCEPTION_EXECUTE_HANDLER)
{
    time_diff = __rdtscp() - time_begin;
    // if time_diff < 4050, it is a mapped address
}
```

1. Generates Page Fault

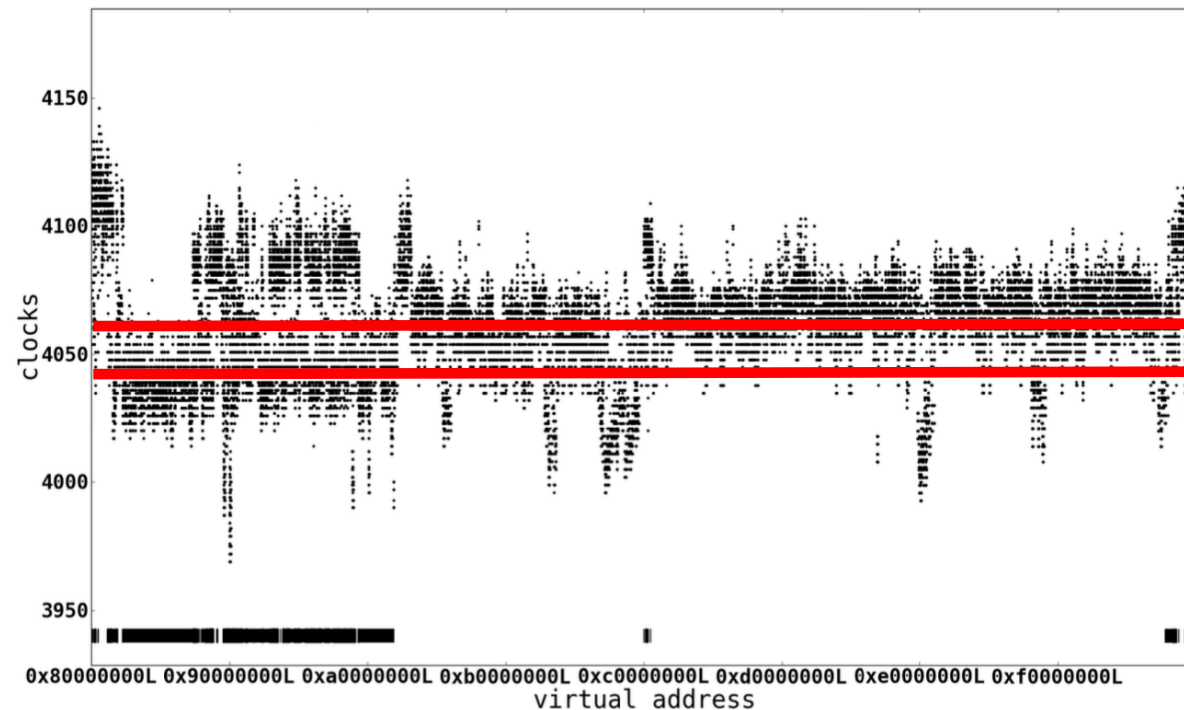
2. CPU generates Page Fault

3. OS handles Page Fault

4. OS calls exception handler

TLB Timing Side Channel

- Result: TLB hit took less than 4050 cycles,
 - While TLB miss took more than that...
- Limitation: Too noisy
 - <1% time difference
 - (~40 within 4000 cycles)
 - OS exception handling is too slow
- Is there any better way?



Outline

- KASLR Background
- TLB Side Channel Attack on KASLR
- **Attacking TLB Side Channel with Intel TSX**
- Attacking various OSes
- Root Cause Analysis
- Discussions
- Conclusion

A More Practical TLB Side Channel Attack on KASLR

- DrK Attack: We present a very practical side channel attack on KASLR
 - De-randomizing Kernel ASLR (this is where DrK comes from)
- Exploit Intel TSX for OS-free exception fallback
 - Accurate: **99%-100%**
 - Fast: **<1 second**
 - OS independent: **Linux, Windows, OS X**
 - Stealthy: **No OS execution path**
 - Cloud: Tested in **Amazon EC2**

Starting From a PoC Example in the Wild

TSX to the rescue

Less noisy

TSX makes kernel address probing much faster and less noisy. If an instruction executed within XBEGIN/XEND block (in usermode) tries to access kernel memory, then no page fault is raised – instead transaction abort happens, so execution never leaves usermode. On my i7-4800MQ CPU, the relevant timings, in CPU cycles, are (minimal/average/variance, 2000 probes, top half of results discarded):

1. access in TSX block to mapped kernel memory: 172 175 2
2. access in TSX block to unmapped kernel memory: 200 200 0
3. access in __try block to mapped kernel memory: 2172 2187 35
4. access in __try block to unmapped kernel memory: 2192 2213 57

TSX Gives Better Precision on Timing Attack

- Access to **mapped** address in TSX: **172** clk
- Access to **unmapped** address in TSX : **200** clk
 - 28 clk (>15%) in timing difference
- Access to **mapped** address in __try: **2172** clk
- Access to **unmapped** address in _try: **2192** clk
 - <1% in timing difference
- Why?

Transactional Synchronization Extension (Intel TSX)

- Traditional Lock

```
pthread_mutex_t *mutex;  
pthread_mutex_lock(mutex);
```

```
// atomic region  
do_atomic_operation();
```

```
pthread_mutex_unlock(mutex);  
// atomic region end
```

1. Block until acquires the lock

2. Atomic region (100% success)

3. Release the lock (finishes atomic region)

Transactional Synchronization Extension (Intel TSX)

- TSX: relaxed but faster way of handling synchronization

```
int status = 0;
if( (status = _xbegin()) == _XBEGIN_STARTED) {
    // atomic region
    try_atomic_operation();
    _xend();
    // atomic region end
}
else {
    // if failed,
    handle_abort();
}
```

1. Do not block, do not use lock

2. Try atomic operation (can fail)

3. If failed, handle failure with abort handler
(retry, get back to traditional lock, etc.)

Transaction Aborts If Exist any of a Conflict

```
int status = 0;
if( (status = _xbegin()) == _XBEGIN_STARTED) {

    // atomic region
    try_atomic_operation();

    _xend();
    // atomic region end
}
else {

    // if failed,
    handle_abort();

}
```

Run If Transaction Aborts

- Condition of Conflict
 - Thread races
 - Cache eviction
 - Interrupt
 - Context Switch (timer)
 - Syscalls
- Exceptions
 - **Page Fault**
 - General Protection
 - Debugging
 - ...

Abort Handler Suppresses Exceptions

```
int status = 0;
if( (status = _xbegin()) == _XBEGIN_STARTED) {

    // atomic region
    try_atomic_operation();

    _xend();
    // atomic region end
}
else {

    // if failed,
    handle_abort();

}
```

Run If Transaction Aborts

- Abort Handler of TSX
 - Suppress all sync. exceptions
 - E.g., page fault
 - **Do not notify OS**
 - Just jump into abort_handler()

No Exception delivery to the OS!
(returns quicker, so less noisy
than `__try __except`)

Exploiting TSX as an Exception Handler

- How to use TSX as an exception handler?

```
uint64_t time_begin, time_diff;
int status = 0;
int *p = (int*)0xffffffff80000000; // kernel address
time_begin = __rdtscp();
if((status = _xbegin()) == _XBEGIN_STARTED) {
    // TSX transaction
    *p; // read access
    // or,
    ((int(*)())p)(); // exec access
}
else {
    // abort handler
    time_diff = __rdtscp() - time_begin;
}
```

1. Timestamp at the beginning

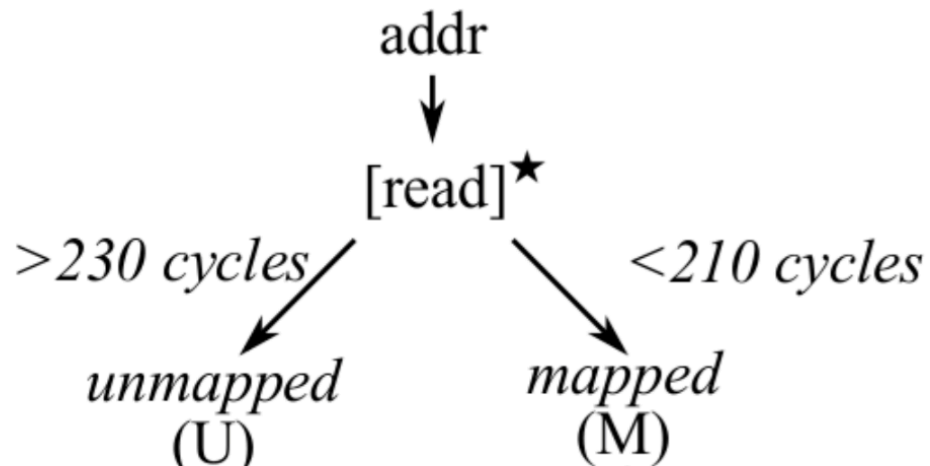
2. Access kernel memory within the TSX region (always aborts)

No OS handling path is involved

3. Measure timing at abort handler

Measuring Timing Side Channel

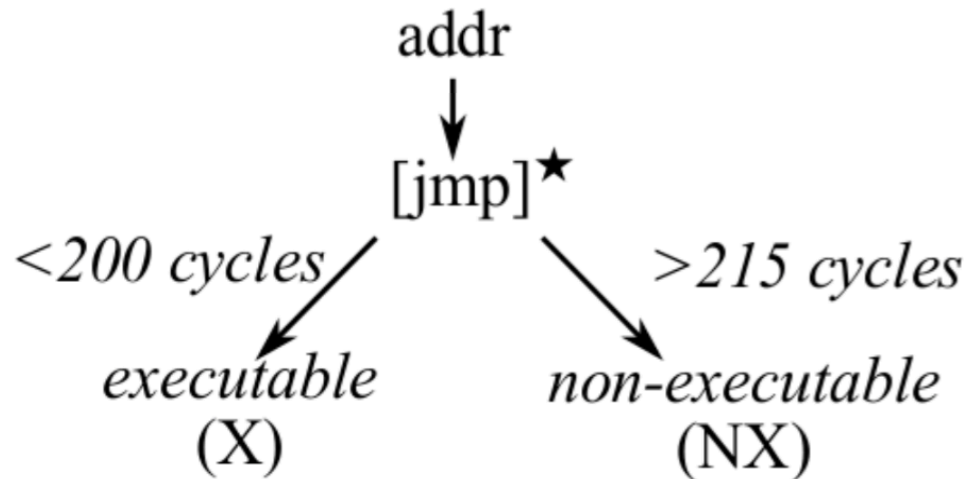
- Access Mapped / Unmapped kernel addresses
 - Attempt **READ** access within the TSX region
 - `mov [rax], 1`



```
def probe(addr):  
    beg = rdtsc()  
    if _xbegin():  
        [mode]★  
    else  
        end = rdtsc()  
    return end - beg
```

Measuring Timing Side Channel

- Access Executable / Non-executable address
 - Attempt **JUMP** access within the TSX region
 - `jmp rax`



```
def probe(addr):  
    beg = rdtsc()  
    if _xbegin():  
        [mode]★  
    else  
        end = rdtsc()  
    return end - beg
```

Demo 1: Timing Difference on M/U and X/NX

Measuring Timing Side Channel

- Mapped / Unmapped kernel addresses
 - Ran 1000 iterations for the probing, minimum clock on 10 runs

Processor	Mapped Page	Unmapped Page
i7-6700K (4.0Ghz)	209	240 (+31)
i5-6300HQ (2.3Ghz)	164	188 (+24)
i7-5600U (2.6Ghz)	149	173 (+24)
E3-1271v3 (3.6Ghz)	177	195 (+18)

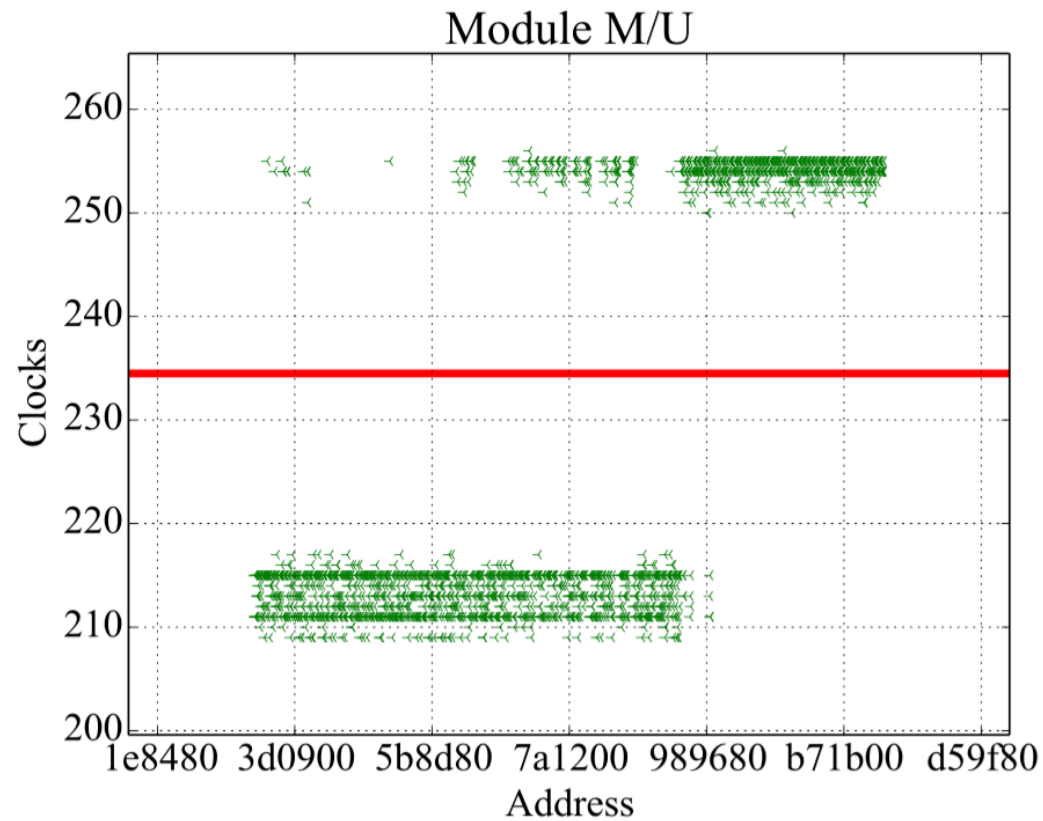
- Much faster than an OS exception handler!
 - 209 versus 4000 cycles
 - Significant time difference: **~15%**

Measuring Timing Side Channel

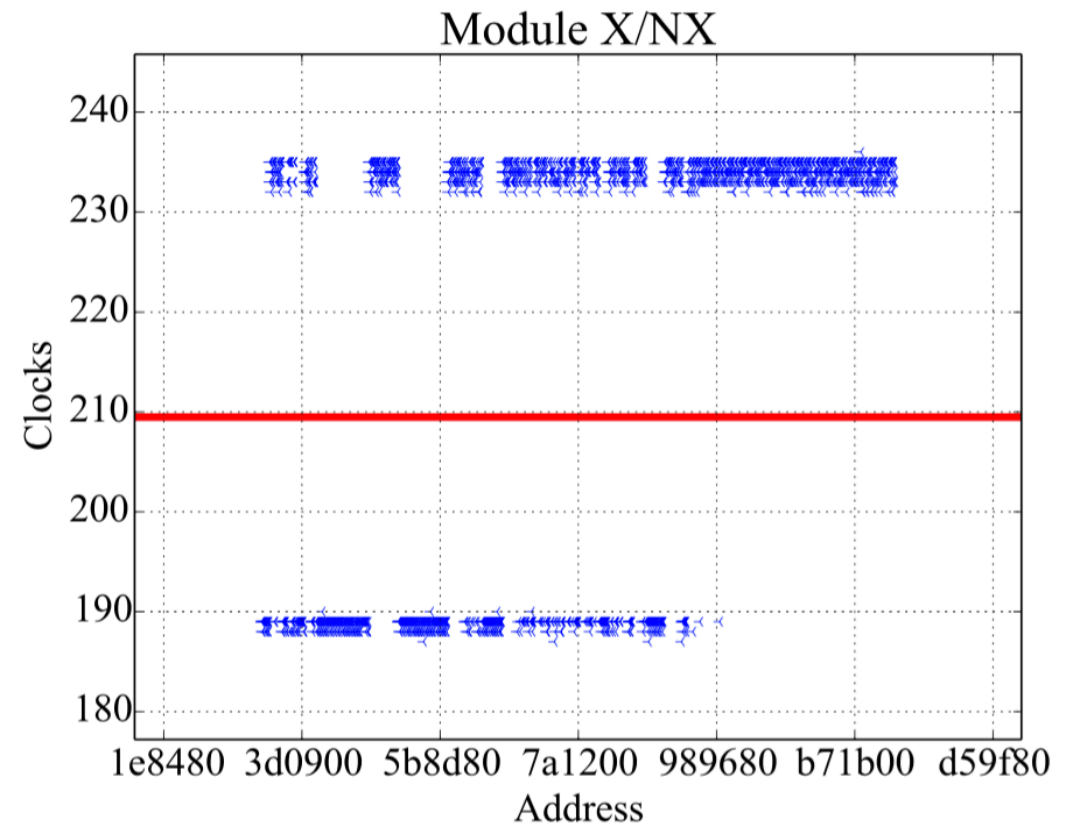
- Executable / Non-executable kernel addresses
 - Ran 1000 iterations for the probing, minimum clock on 10 runs

Processor	Executable Page	Non-exec Page
i7-6700K (4.0Ghz)	181	226 (+45)
i5-6300HQ (2.3Ghz)	142	178 (+36)
i7-5600U (2.6Ghz)	134	164 (+30)
E3-1271v3 (3.6Ghz)	159	189 (+30)

Clear Timing Channel



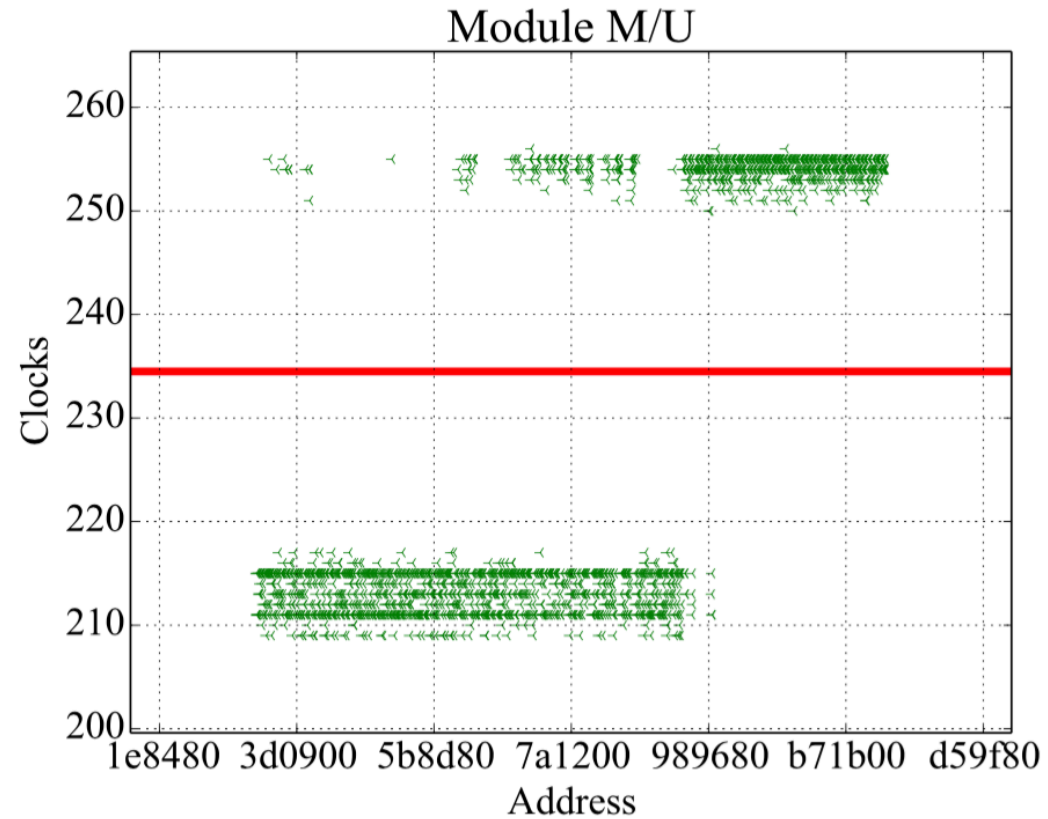
(a) Mapped vs. Unmapped



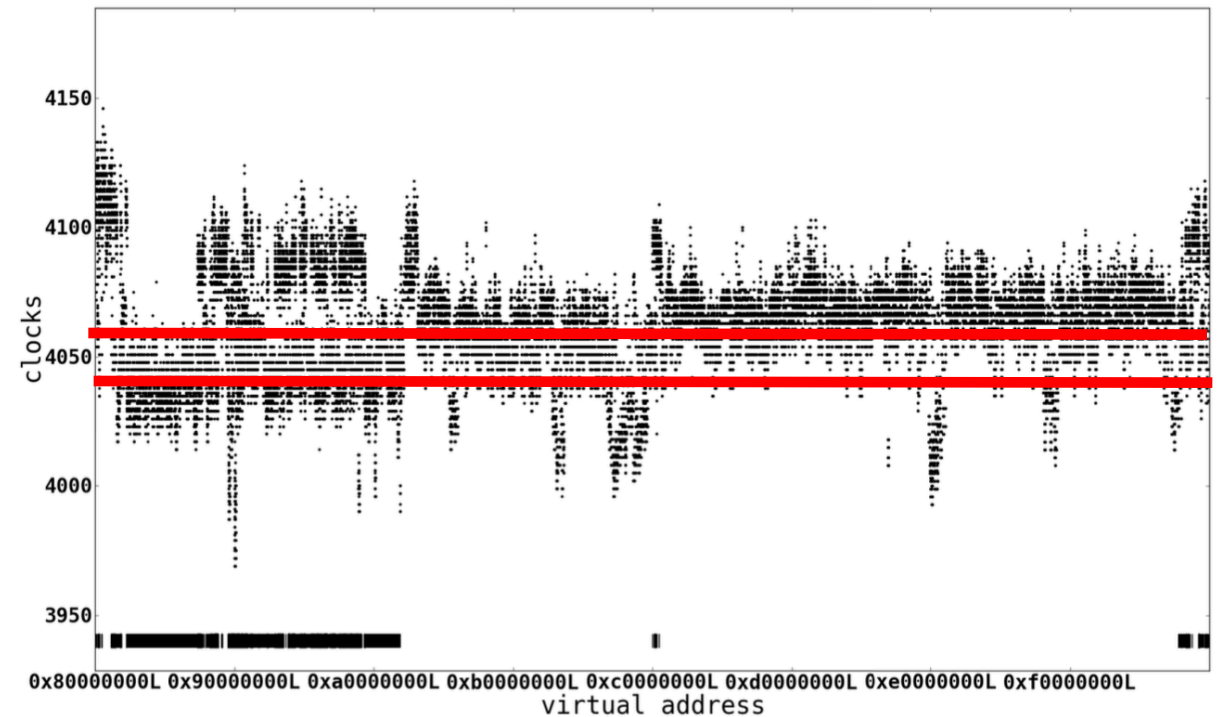
(b) Executable vs. Non-executable

Clear separation between different mapping status!

TSX vs SEH



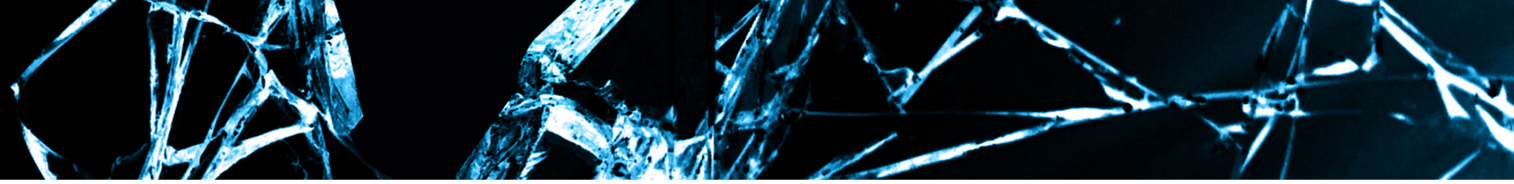
(a) Mapped vs. Unmapped



Clear separation between different mapping status!

Outline

- KASLR Background
- TLB Side Channel Attack on KASLR
- Attacking TLB Side Channel with Intel TSX
- **Attacking various OSes**
- Root Cause Analysis
- Discussions
- Conclusion



Attack on Various OSes

- Demo Targets
 - Full attack
 - Linux, Windows, and Linux in Amazon EC2
 - Probe each page of kernel/drivers (>6,000 in Linux, >34,000 in Windows)
 - Compare its permission to page table to get the accuracy
 - Detecting Modules Location
 - Based on section size (X/NX/U), detect the exact location of kernel module
 - Finding ASLR slide
 - OS X

Attack on Linux

- OS Settings
 - Kernel 4.6.0, running with Ubuntu 16.04 LTS
 - Added bootarg 'kaslr'
 - Enabled with CONFIG_X86_PTDUMP=y (just for ground truth)
- Available Slots
 - Kernel: 64 slots
 - 0xffffffff80000000 – 0xffffffffc0000000 (2MB page)
 - Module: 1,024 slots
 - 0xffffffffc0000000 – 0xffffffffc0400000 (4KB page)

Demo 2: Full Attack on Linux

Result

- Achieved 100% accuracy across 3 different CPUs
 - Took 0.45-0.67s for probing 6,147 pages.
- Detecting Modules
 - From size signature, detected 29 modules among 80 modules.

Attack on Windows

- OS Settings
 - Windows 10, 10.0.10586
 - Available Slots
 - Kernel: 8,192 slots
 - 0xfffff80000000000 - 0xfffff80400000000 (2 MB pages)
 - Drivers: 8,192 slots
 - 0xfffff80000000000 - 0xfffff80400000000 (4 KB pages, aligned with 2 MB)

Result

- 100% of accuracy for the kernel (ntoskrnl.exe)
- 100% of accuracy for detecting M/U for the drivers
- 99.28% of accuracy for detecting X/NX for drivers
 - Some areas in driver are dynamically deallocated
 - Misses some 'inactive' pages
- Detecting Modules
 - From size signature, detected 97 drivers among 141 drivers

Attack on OS X

- OS Settings
 - OS X El Capitan 10.11.4
 - Available Slots
 - Kernel: 256 slots
 - 0xffffffff800000000 - 0xffffffff802000000 (2 MB pages)
- Result
 - Took 31 ms on finding ASLR slide (100% accuracy for 10 times)

Attack on Amazon EC2

- OS Settings
 - Kernel 4.4.0, running with Ubuntu 14.04 LTS
 - Added bootarg 'kaslr'
 - Enabled with CONFIG_X86_PTDUMP
- Available Slots
 - Kernel: 64 slots
 - 0xffffffff80000000 – 0xffffffffc0000000 (2MB page)
 - Module: 1,024 slots
 - 0xffffffffc0000000 – 0xffffffffc0400000 (4KB page)

Result Summary

- Linux: 100% of accuracy around 0.5 second
- Windows: 100% for M/U in 5 sec, 99.28% for X/NX for 45 sec
- OS X: 100% for detecting ASLR slide, in 31ms
- Linux on Amazon EC2: 100% of accuracy in 3 seconds

Outline

- KASLR Background
- TLB Side Channel Attack on KASLR
- Attacking TLB Side Channel with Intel TSX
- Attacking various OSes
- **Root Cause Analysis**
- Discussions
- Conclusion

Timing Side Channel (M/U)

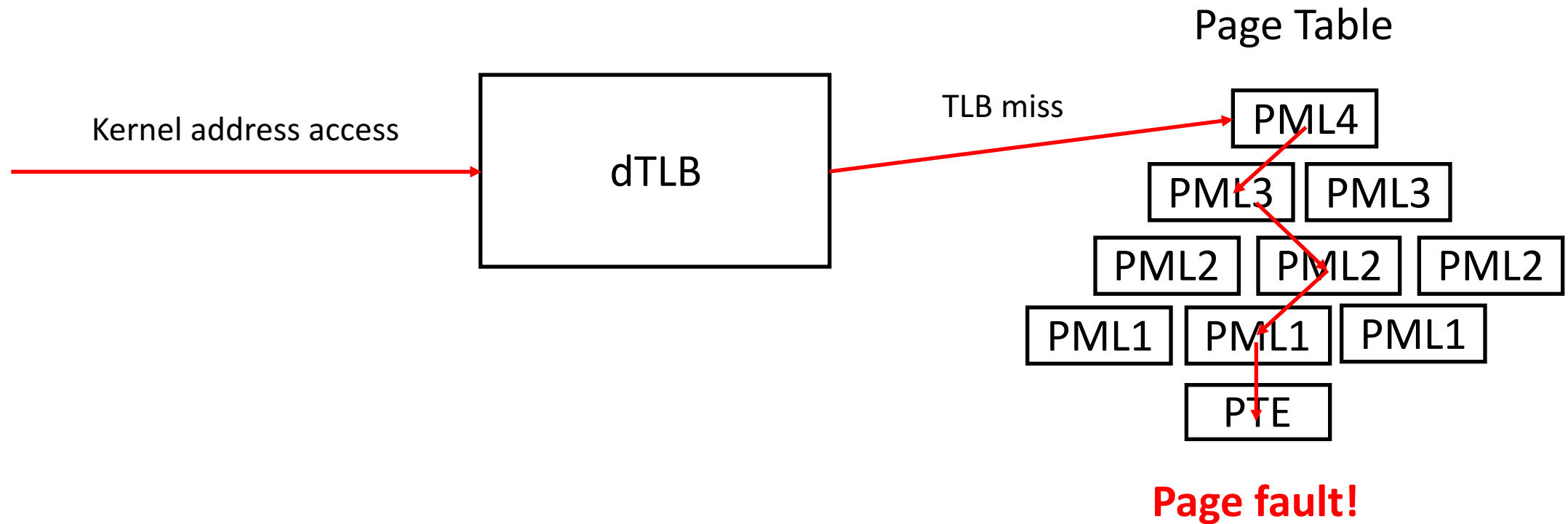
- For Mapped / Unmapped addresses
 - Measured performance counters (on 1,000,000 probing)

Perf. Counter	Mapped Page	Unmapped Page	Description
dTLB-loads	3,021,847	3,020,243	
dTLB-load-misses	84	2,000,086	TLB-miss on U
Observed Timing	209 (fast)	240 (slow)	

- dTLB hit on mapped pages, but not for unmapped pages.
 - Timing channel is generated by dTLB hit/miss

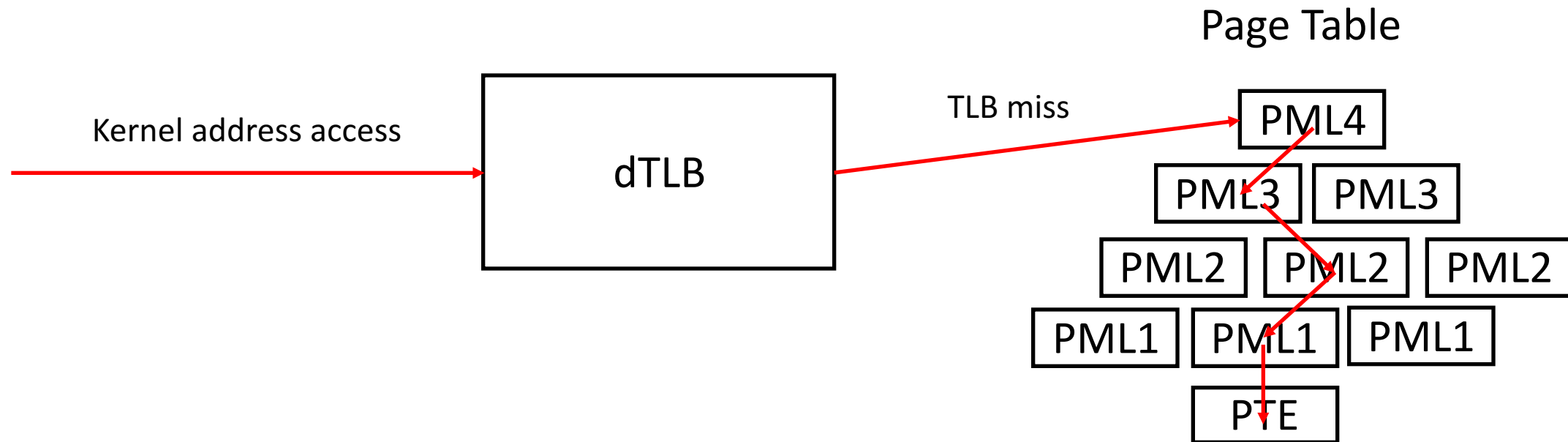
Path for an Unmapped Page

On the first access



Path for an Unmapped Page

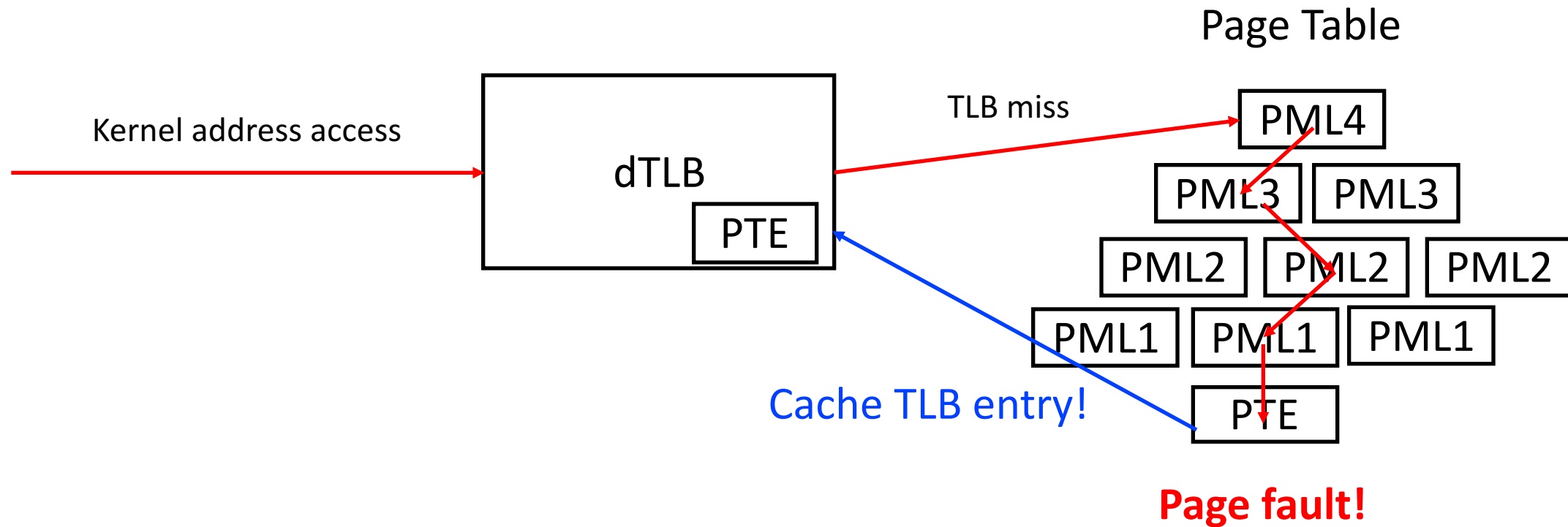
On the Second access



Always do page table walk (**slow**)

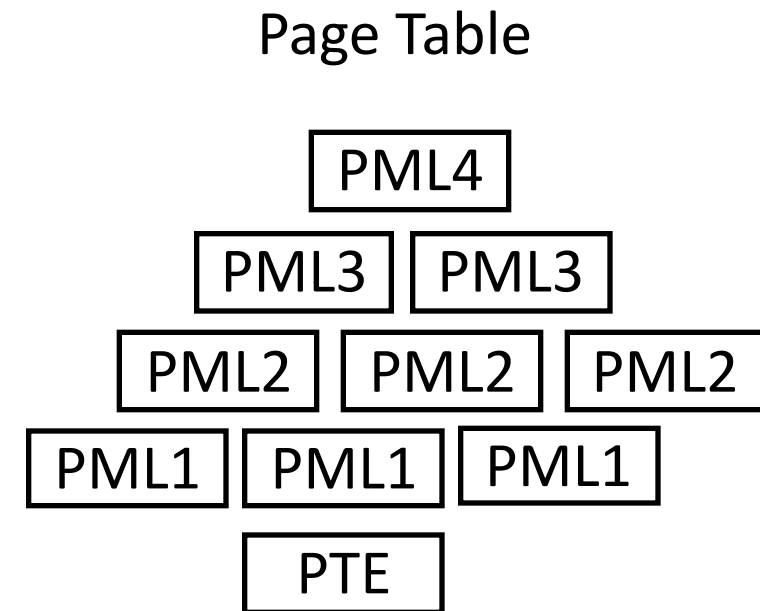
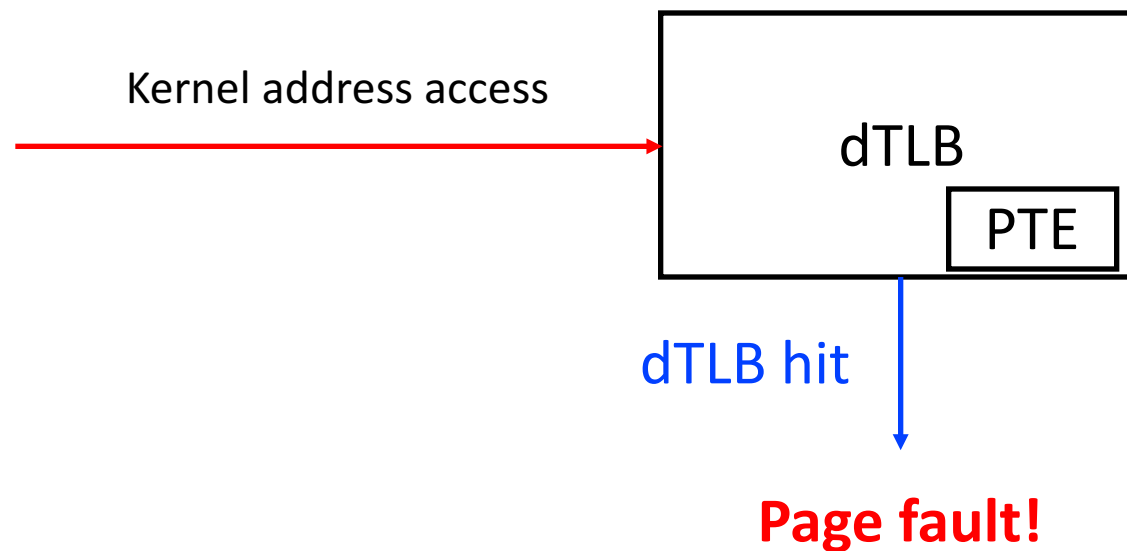
Path for a mapped Page

On the first access



Path for a mapped Page

On the second access



No page table walk on the second access (**fast**)

Root-cause of Timing Side Channels (M/U)

- For Mapped / Unmapped addresses

Fast Path (Mapped)	Slow Path (Unmapped)
1. Access a Kernel address 2. dTLB hits 3. Page fault!	1. Access a Kernel address 2. dTLB misses 3. Walks through page table 4. Page fault!
Elapsed cycles: 209	Elapsed cycles: 240

- Caching at dTLB generates timing side channel

Timing Side Channel (X/NX)

- For Executable / Non-executable addresses
 - Measured performance counters (on 1,000,000 probing)

Perf. Counter	Exec Page	Non-exec Page	Unmapped Page
iTLB-loads (hit)	590	1,000,247	272
iTLB-load-misses	31	12	1,000,175
Observed Timing	181 (fast)	226 (slow)	226 (slow)

- Point #1: iTLB hit on Non-exec, but it is slow (226) why?
- iTLB is not the origin of the side channel.

Timing Side Channel (X/NX)

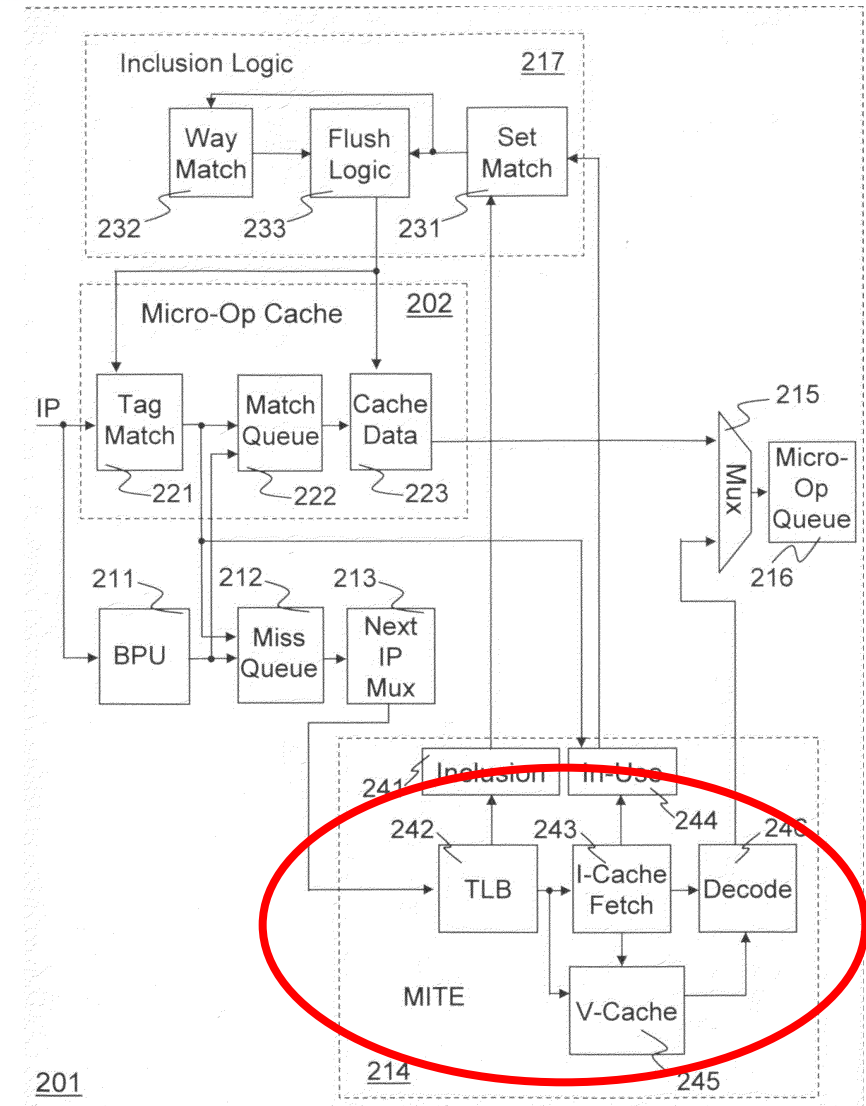
- For Executable / Non-executable addresses
 - Measured performance counters (on 1,000,000 probing)

Perf. Counter	Exec Page	Non-exec Page	Unmapped Page
iTLB-loads (hit)	590	1,000,247	272
iTLB-load-misses	31	12	1,000,175
Observed Timing	181 (fast)	226 (slow)	226 (slow)

- Point #2: iTLB does not even hit on Exec page, while NX page hits iTLB
- iTLB is not involved in the fast path

Intel Cache Architecture

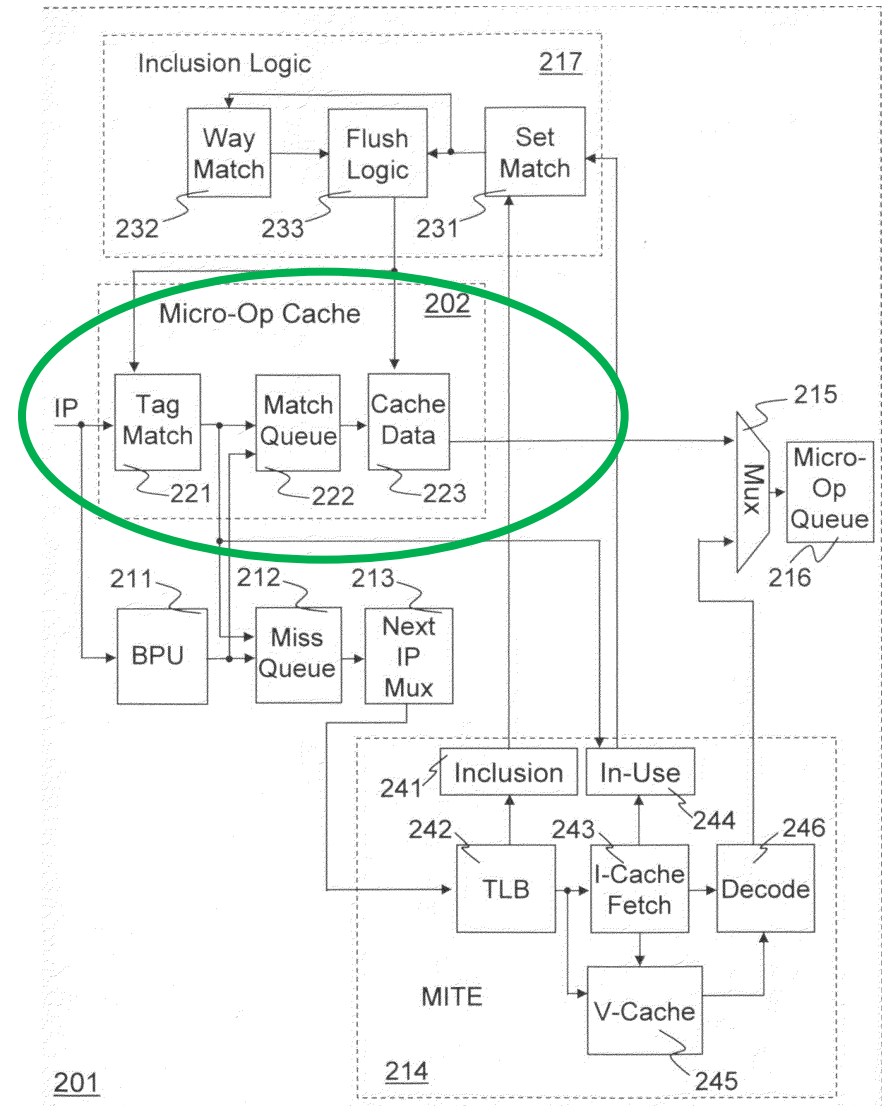
- L1 instruction cache
 - Virtually-indexed, Physically-tagged cache (requires TLB access)
 - Caches actual opcode / data content of the memory



From the patent **US 20100138608 A1**,
registered by Intel Corporation

Intel Cache Architecture

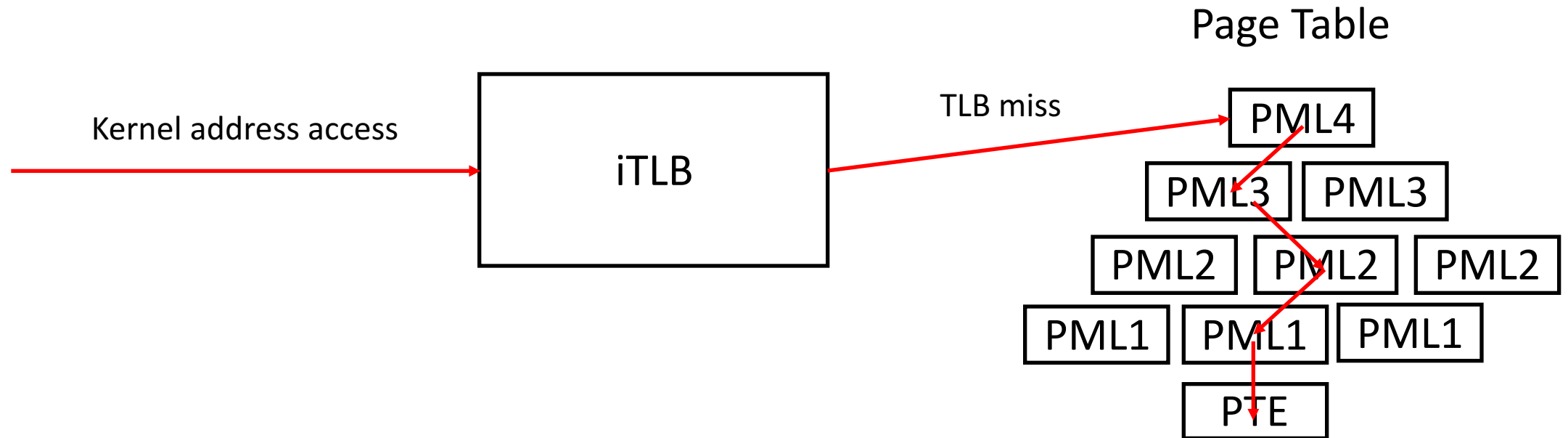
- Decoded i-cache
 - An instruction will be decoded as micro-ops (RISC-like instruction)
 - Decoded i-cache stores micro-ops
 - Virtually-indexed, Virtually-tagged cache (no TLB access)



From the patent **US 20100138608 A1**,
registered by Intel Corporation

Path for an Unmapped Page

On the Second access, **226** cycles

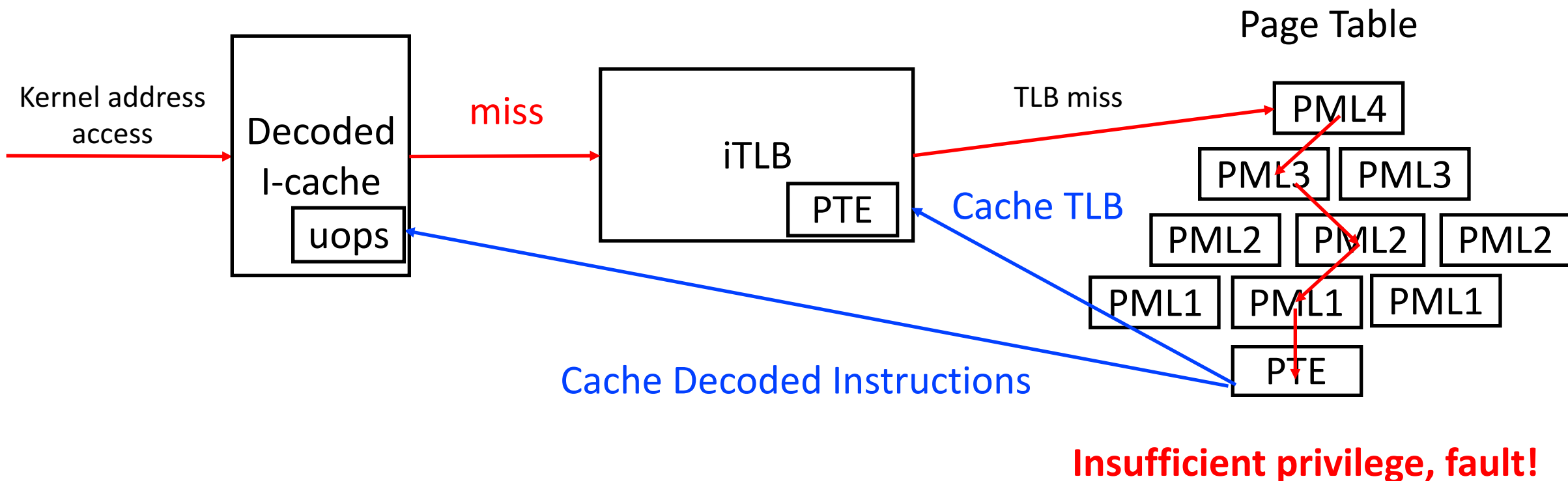


Page fault!

Always do page table walk (**slow**)

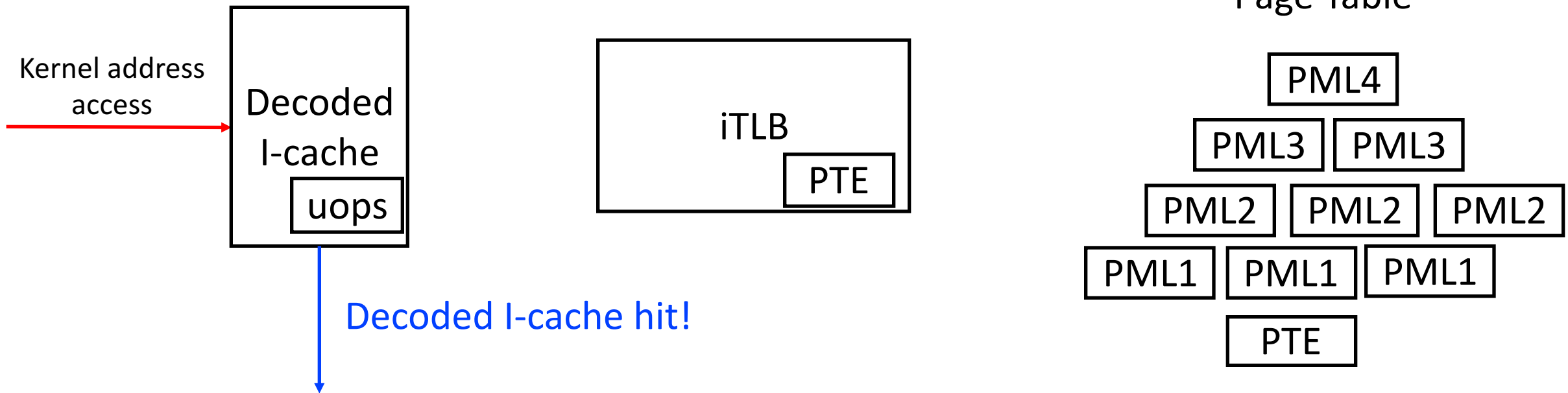
Path for an Executable Page

On the first access



Path for an Executable Page

On the second access, **181** cycles

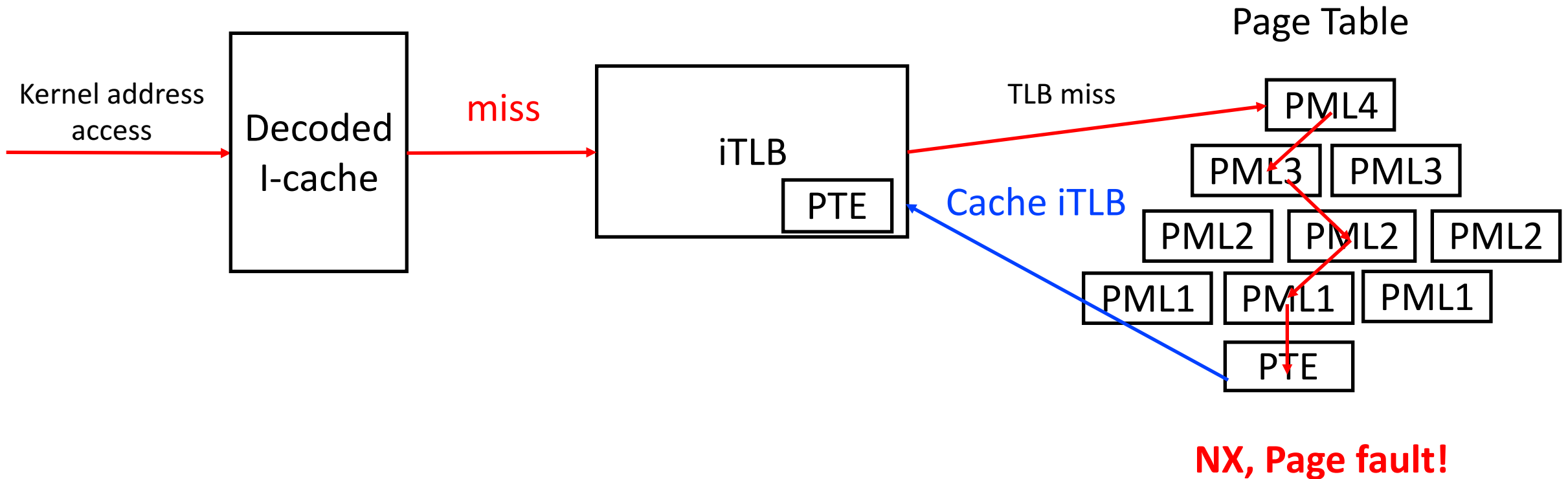


Insufficient privilege, fault!

No TLB access, No page table walk (**fast**)

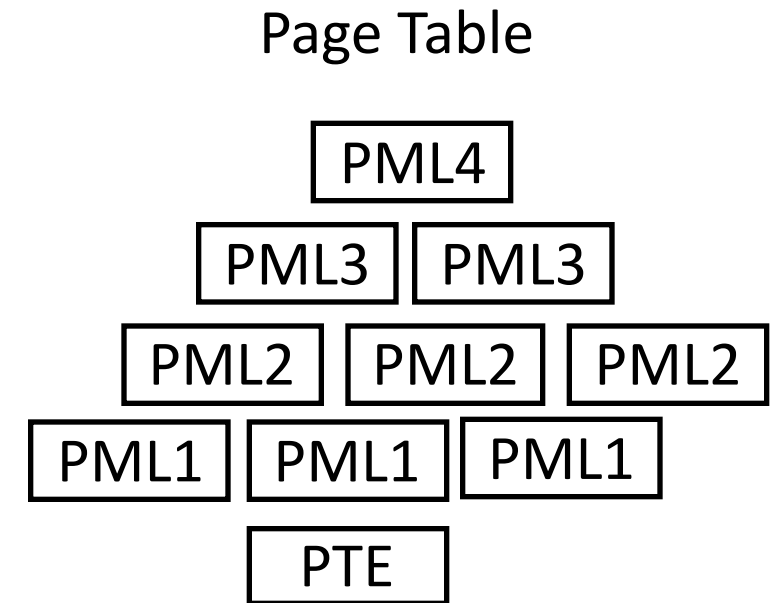
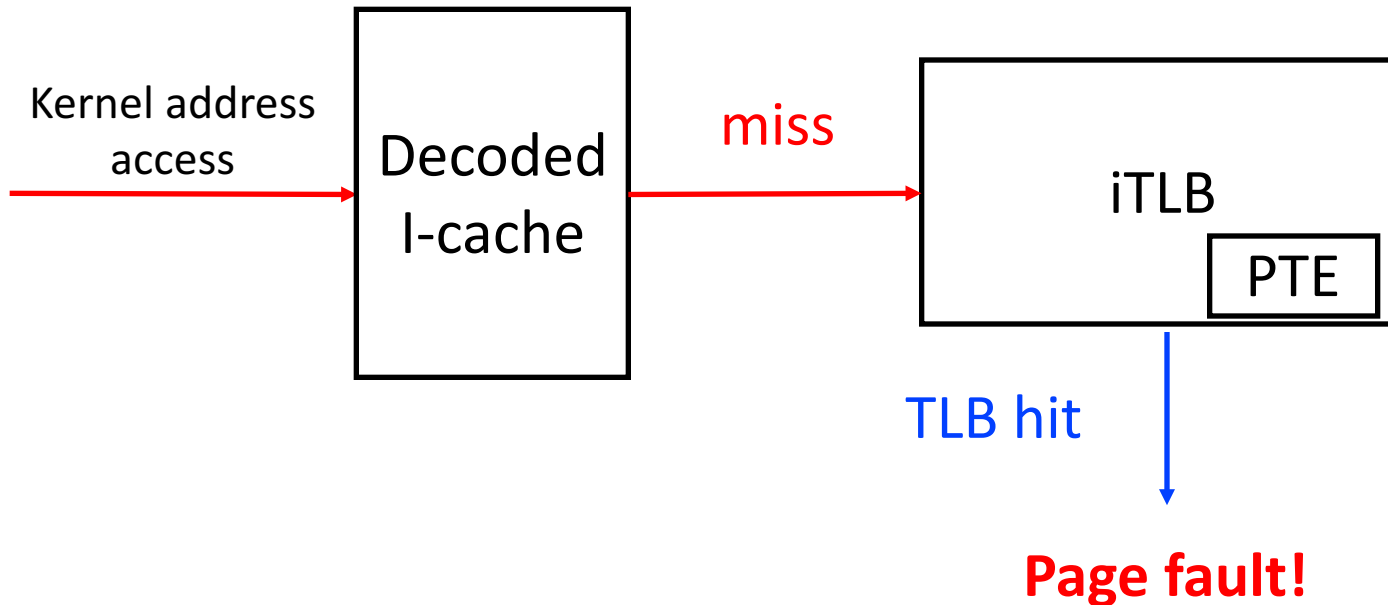
Path for a non-executable, but mapped Page

On the first access



Path for a Non-executable, but mapped Page

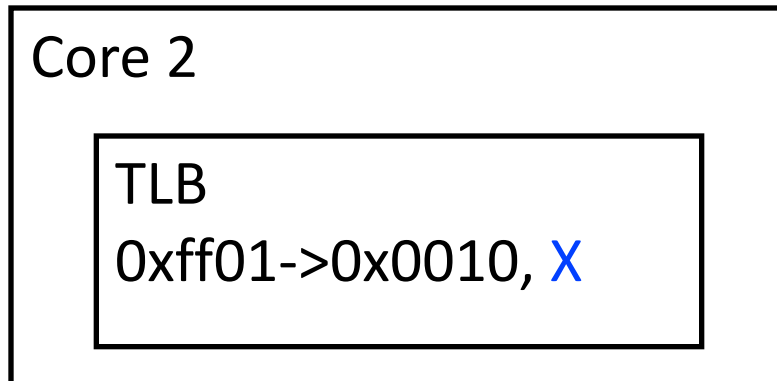
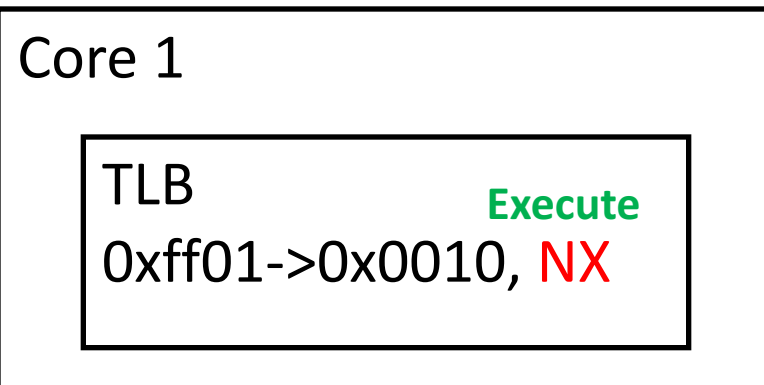
On the second access, **226** cycles



If no page table walk, it should be faster than unmapped (**but not!**)

Cache Coherence and TLB

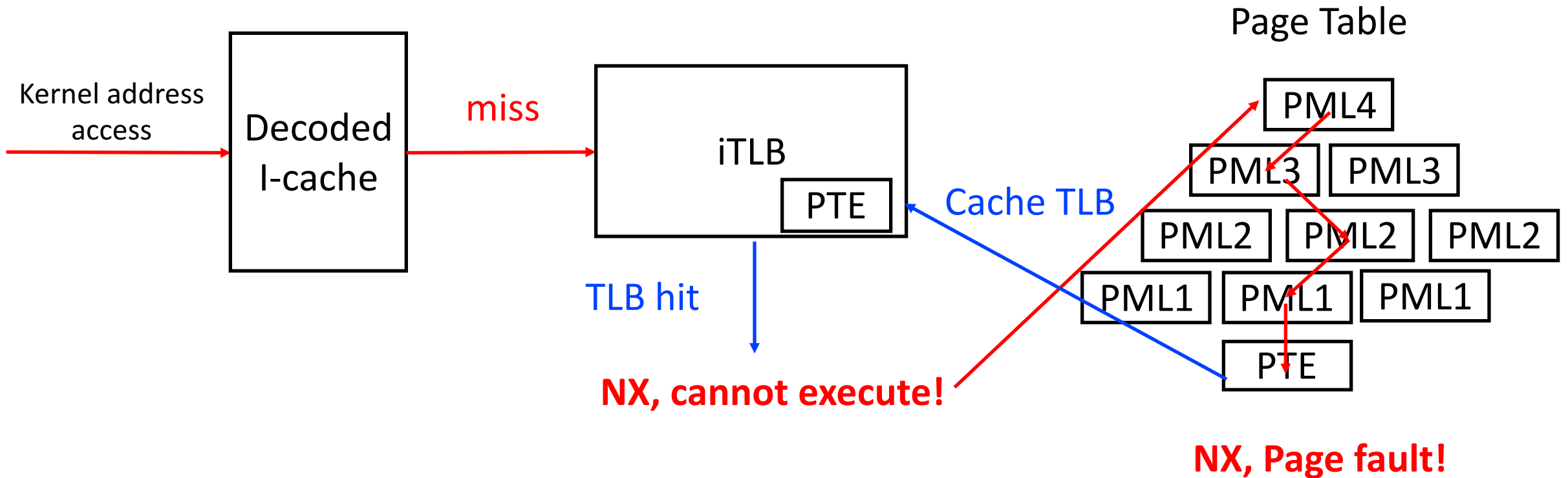
- TLB is not a coherent cache in Intel Architecture



1. Core 1 sets 0xff01 as **Non-executable** memory
2. Core 2 sets 0xff01 as **Executable** memory
No coherency, **do not** update/invalidate TLB in Core 1
3. Core 1 try to execute on 0xff01 -> Page fault by NX
4. Core 1 **must walk through the page table**
The page table entry is **X**, update TLB, then **execute**!

Path for a Non-executable, but mapped Page

On the second access, **226** cycles

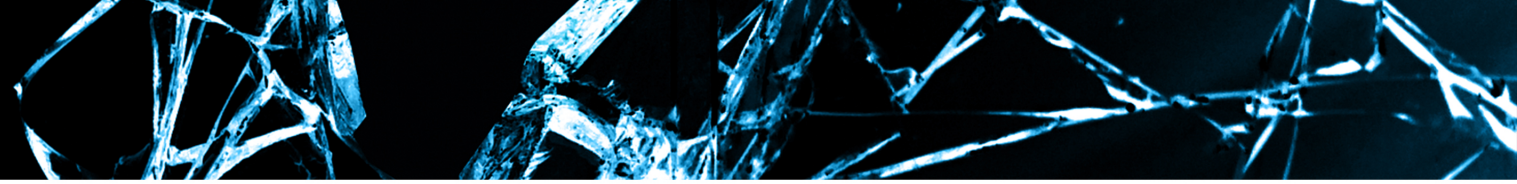


Root-cause of Timing Side Channel (X/NX)

- For eXecute / non-executable addresses

Fast Path (X)	Slow Path (NX)	Slow Path (U)
1. Jmp into the Kernel addr 2. Decoded I-cache hits 3. Page fault!	1. Jmp into the kernel addr 2. iTLB hit 3. Protection check fails, page table walk. 4. Page fault!	1. Jmp into the kernel addr 2. iTLB miss 3. Walks through page table 4. Page fault!
Cycles: 181	Cycles: 226	Cycles: 226

- Decoded i-cache generates timing side channel



Analysis Summary

- dTLB caching makes faster fault on mapped address
 - Mapped: PTE cached in dTLB
 - Unmapped: PTE is not cached in dTLB, requires page table walk
- Decoded I-cache makes faster fault on executable address
 - Executable: Decoded i-cache hits, no iTLB access, no page table walk
 - Non-executable: iTLB hits, but requires page table walk
 - Unmapped: always requires page table walk

Outline

- KASLR Background
- TLB Side Channel Attack on KASLR
- Attacking TLB Side Channel with Intel TSX
- Attacking various OSes
- Root Cause Analysis
- **Discussions**
- Conclusion

Discussions: Controlling Noise

- Dynamic frequency scaling (SpeedStep, TurboBoost) changes the return value of `rdtscp()`.
 - Run busy loops to make CPU run as full-throttle
- Hardware interrupts and cache conflicts also abort TSX.
 - Probe multiple times (e.g., 2-100) and take the minimum

Discussions: Increasing Coverttness

- OS never sees page faults
 - TSX suppresses the exception
- Possible traces: performance counters
 - High count on dTLB/iTLB-miss
 - Normal programs sequentially accessing huge memory could behave similarly.
 - High count on tx-aborts or CPU time
 - Attackers could slow down the probing rate (e.g., 5 min, still fast)

Discussions: Countermeasures?

- Modifying CPU to eliminate timing channels
 - Difficult to be realized ☹
- Using separated page tables for kernel and user processes
 - High performance overhead (~30%) due to frequent TLB flush
- Fine-grained randomization
 - Difficult to implement and performance degradation
- Coarse-grained timer?
 - Always suggested, but no one adopts it.

Outline

- KASLR Background
- TLB Side Channel Attack on KASLR
- Attacking TLB Side Channel with Intel TSX
- Attacking various OSes
- Root Cause Analysis
- Discussions
- **Conclusion**

Conclusion

- TSX can break KASLR of commodity OSes.
 - Ensure accuracy, speed, and covertness
- Timing side channel is caused by hardware, independent to OS.
 - dTLB (for Mapped & Unmapped)
 - Decoded I-cache (for eXecutable / non-executable)
- We consider potential countermeasures against this attack.